ACICP302ICS

Module 1 Practical Lab Assessment Brief

Source-Code-Guided OT Simulation Lab

Document detail	Information
Module	Module 1 Integrated ICS Network Security & Attacks: Analyze, Test & Recover the Supplied OT Simulation
Activity title	Analyze, Test & Defend the Supplied Modbus/TCP OT Simulation
Assessment workflow	Governed Practical Laboratories evidence workflow
Maximum score / pass	100 marks / 50% practical pass threshold
Contribution category	Practical Laboratories
Submission mode	Private learner GitHub repository + associated evidence

Read this first

This is a source-code-guided practical. Students use and analyze the supplied loopback-safe simulation; they are not required to build a replacement simulator. Use only the canonical student source and follow the safety boundary in this document.

International Cybersecurity and Digital Forensics Academy (ICDFA) | Version 2.0 | 05 October 2026

1. Scenario

You are an OT security analysis team assigned a small simulated process based on the course-provided Modbus/TCP source. Your task is to inspect and run the supplied loopback-safe simulation in an isolated Ubuntu virtual machine, establish normal operation, analyze protocol and guard behavior, complete controlled demonstrations, assess limitations and verify recovery. You are not required to build a replacement simulator or guard.

The source is supplied; the analysis is yours

Use the canonical course source at `labs/ot-security/student-lab-source/`. The archived original at `labs/ot-security/archive-original/` contains unsafe network defaults and must not be executed. Submit your own observations, captures, risk analysis, report and evidence not the course repository as if it were your project.

2. Objectives

- Safely locate and verify the canonical course source, record its commit and run it only inside the assigned Ubuntu VM on loopback.
- Explain the supplied architecture, process model, Modbus/TCP data points and direct/guarded paths using source and runtime evidence.
- Capture normal loopback traffic, identify a request/response and connect packet fields to process state.
- Document the local service exposure and process-related risks within the authorized scope.
- Analyze the supplied guard's checks, events, assumptions and limitations without misrepresenting capabilities.
- Plan and run at least three approved bounded demonstrations, compare expected and observed results, and verify recovery.
- Submit a private evidence repository with traceable, learner-created artifacts and defend conclusions accurately.

3. Full student briefing

Before beginning, obtain access to the private course repository from the instructor or use the instructor-provided archive. Read the Student Lab Guide. Prepare a fresh, instructor-approved Ubuntu guest VM and clean snapshot. Do not use bridged networking or port forwarding. Complete approved repository/package downloads during temporary setup access, then disconnect the VM's network adapter before running the simulation.

Run only `labs/ot-security/student-lab-source/`. It is the canonical course copy configured to use `127.0.0.1`. Never execute files in `labs/ot-security/archive-original/`; the archived bundle uses a non-loopback destination (`192.168.2.139`) and broad listener binds (`0.0.0.0`). If the loopback check fails or scope is unclear, stop and contact the instructor. Do not edit the code to force a pass.

Complete Activities 00–08 in order unless the instructor explicitly waives an activity: preflight and safety; source architecture and normal baseline; supplied Modbus/TCP data model; loopback packet analysis; local exposure mapping; supplied guard analysis; controlled validation using at least three approved demonstration scripts; recovery/evidence/report/defence; and the full source-code lab run. Use supplied script defaults unchanged and run one demonstration at a time. Availability/load demonstrations require explicit instructor approval.

Submit your own work through the governed Practical Laboratories workflow: a private learner/team GitHub evidence repository and associated evidence links, with matching commit/branch/file references. Do not submit the course source repository as your own. Cite its URL and exact commit; clearly separate instructor-provided source from your analysis. Maximum marks are protected after the first submission. Follow the opening/due dates shown in the LMS (WAT).

4. Course source and access

Course repository: https://github.com/icdfa/ACICP302ICS-Network-Security-and-Attacks

Canonical student source: https://github.com/icdfa/ACICP302ICS-Network-Security-and-Attacks/tree/main/labs/ot-security/student-lab-source

Complete Ubuntu VM guide: https://github.com/icdfa/ACICP302ICS-Network-Security-and-Attacks/blob/main/labs/ot-security/STUDENT_LAB_GUIDE.md

Private repository access

The course repository is private. The instructor/course owner must grant access or provide an approved archive through the LMS. Do not make the course repository public solely to avoid access setup. Learner evidence repositories should remain private unless the instructor specifies otherwise.

5. Required activities and evidence

Activity 00 - Lab preflight and safety

Prepare the assigned Ubuntu VM and verify the exact course source and authorization boundary before starting any simulation process.

Expected tasks: Obtain access to the private course repository from the instructor, or use the approved archive. Read the Student Lab Guide. Create/restore the instructor-assigned clean VM snapshot. Confirm the adapter is not Bridged and port forwarding is absent; disconnect the adapter after approved downloads and dependency setup. Locate `labs/ot-security/student-lab-source/`. Do not execute files in `labs/ot-security/archive-original/`. Verify the student code uses `127.0.0.1` for clients and service binds. Stop and notify the instructor if the check fails; do not edit the source to make it pass. Record the course source commit, VM/snapshot, date and WAT time, authorized scope, stop condition and recovery method. Begin the evidence log and private learner evidence workspace.

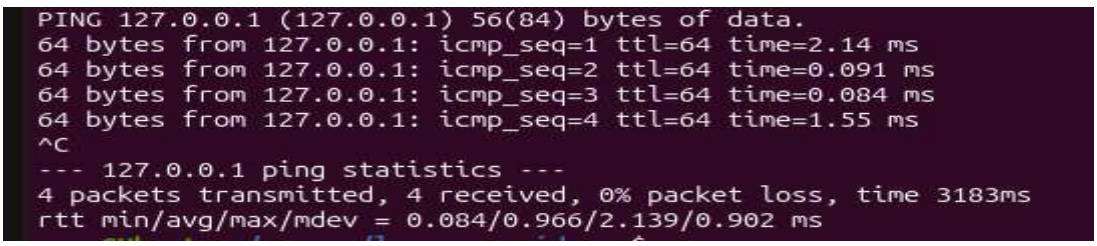
Evidence: Completed preflight/evidence log; source path and commit; loopback check; VM/snapshot details; topology draft; signed safety declaration.

Here is a clean, ready-to-use template populated with realistic lab example values (such as an OpenPLC/FUXA HMI test environment) that you can copy, edit, and fill out for your submission.

1. VM & Snapshot Details
   - Virtual Machine Name: ICDFA-SCADA-Server
   - Virtualization Software: VirtualBox
   - Snapshot Name: Clean_Baseline
   - Snapshot Date/Time: 2026-10-10 03:42 PM WAT

2. Source Path & Version Control
   - Source Path:cd /course/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source
   - Git Commit Hash: 2a66f9bcf253eb3fe31c68ad52794cd535421da8

3. Loopback & Network Check Result
   - Command Executed: ping -c 4 127.0.0.1



SAFETY & ETHICS DECLARATION

I hereby declare that all technical operations, protocol analysis, and security testing conducted for this assignment were executed strictly within an isolated, non-production virtual machine laboratory environment (Ubuntu-OT-Security-Lab-VM01).

- No active network probing, packet injection, or vulnerability scanning was directed at real-world production systems, external networks, or unauthorized hardware devices.
- All network interfaces were verified to ensure host-only / private loopback isolation prior to test execution.
- I have complied with all applicable institutional academic integrity guidelines and lab safety protocols.

Student Name: Isaac Oyero

Student ID: [Insert Your Student ID]

Date: October 12, 2026

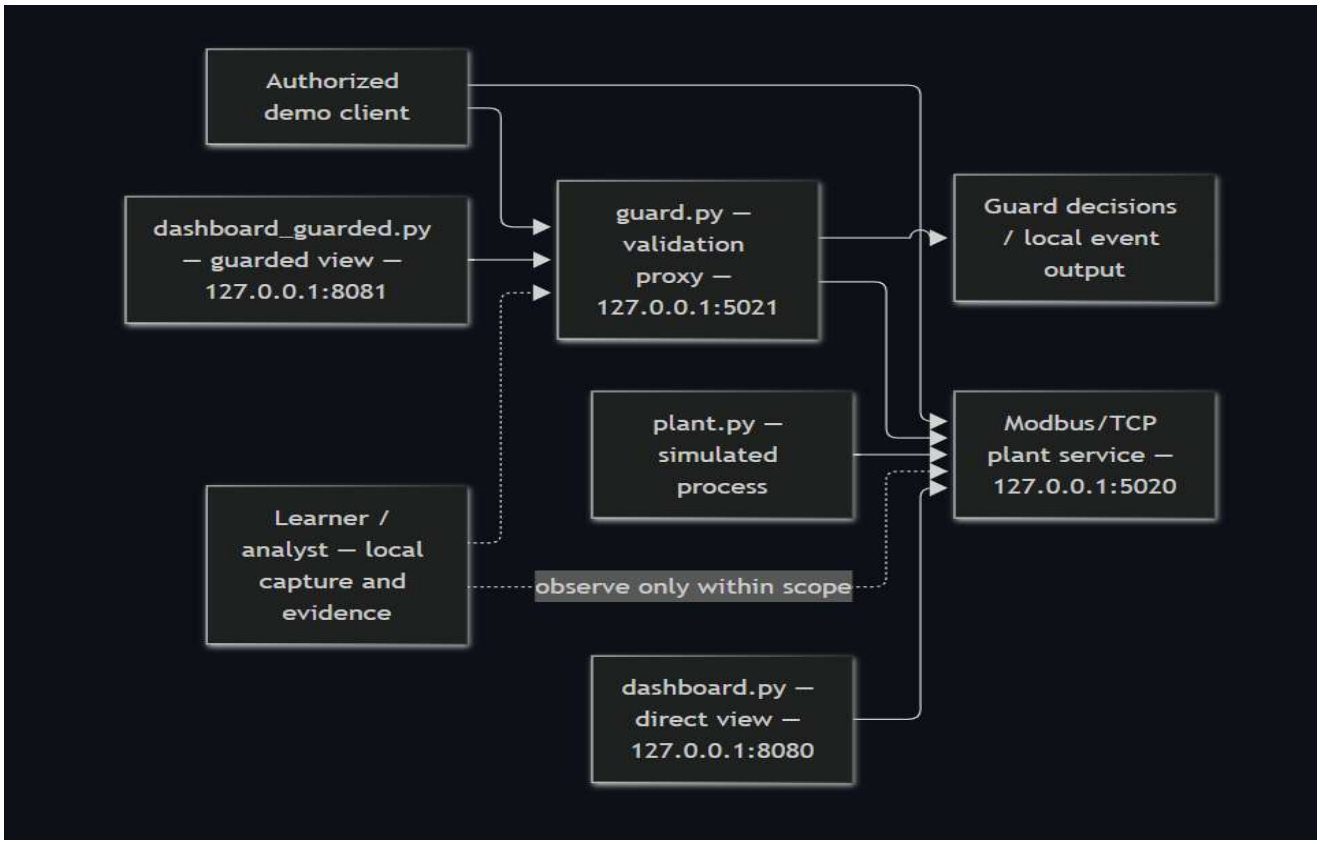
Signature: Isaac Oyero

Activity 01 - Architecture and plant baseline

Understand the supplied simulation architecture and establish a learner-observed normal baseline; do not build a replacement plant.

Expected tasks: Read `docs/reference-architecture.md` and inspect the files in `student-lab-source/plant_sim/`. Trace the simulated plant, Modbus/TCP service, direct dashboard, guard, guarded dashboard and supplied client scripts. Annotate the architecture with roles and data flow. Follow the guide to start only normal services. Observe the process and dashboards before any demonstration. Record normal values, update behavior and any differences between direct and guarded views; stop services as directed.

Evidence: Annotated topology/data-flow diagram; source references; screenshots of normal dashboards; baseline observations and evidence-log entry.



IMG: Annotated topology/data-flow diagram.

Component-role notes based on source inspection:

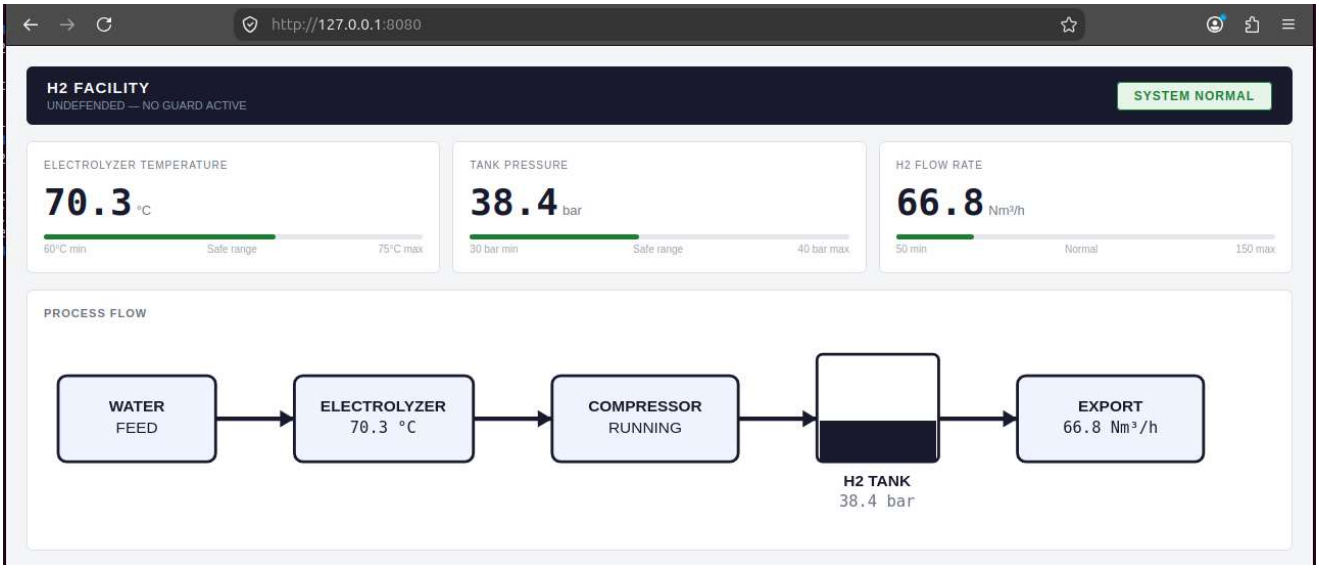
plant.py: The simulated unguarded modbus plant, runs on port 5020. It returns a structured log stream in the terminal when running.

guard.py: The simulated guarded modbus plant, runs on port 5021. It intercept, filter, and manage incoming traffic before it reaches the backend plant service.

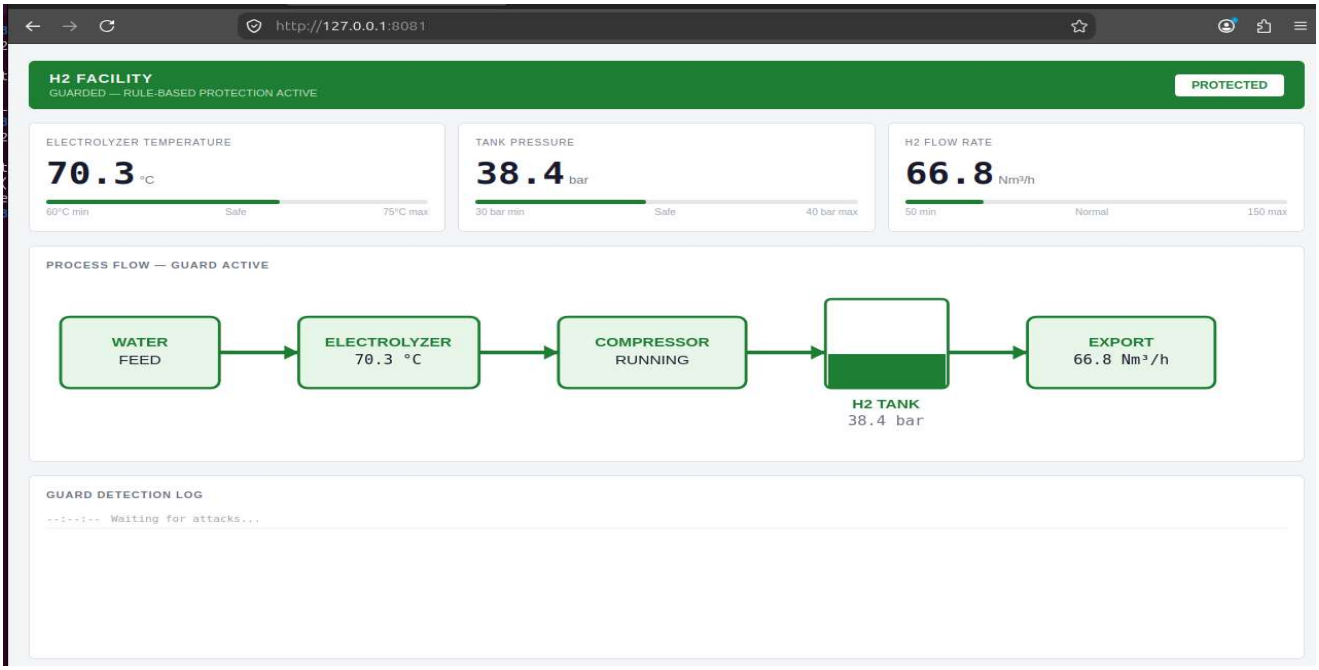
dashboard.py: The dashboard shows the H2 facility web interface operating without a security guard, displaying real-time plant telemetry (electrolyzer temperature, tank pressure, and flow rate) across the process flow without any active security logs.

dashboard_guarded.py: The dashboard shows the same H2 facility interface under rule-based protection, featuring an active "Protected" status banner, green-themed process flow styling, and an integrated "Guard Detection Log" monitoring panel.

Screenshot of normal dashboards



IMG: normal unguarded dashboard.



IMG: normal guarded dashboard.

Activity 02 - Modbus/TCP data model and normal operations

Document the data model implemented by the supplied simulation and relate its definitions to observed normal behavior.

Expected tasks: Inspect the source definitions for process values, register addresses, units/scales, normal bounds, status/alarm semantics and update behavior. Create a data-point table. Mark unknowns as unknown; do not guess. Run the normal local simulation and verify selected reads using only the guide's documented local path. Explain the client/server roles and a normal request/response, linking values to the dashboard. Note any source/comment/display mismatch.

Evidence: Learner-prepared register map; source references; normal-read/dashboard evidence; concise transaction explanation.

Register Map

Register	Variable	Unit	Normal Read	Evidence
0	Temperature	Celcius (^{0}C)	(600, 750) # tenths of a degree C	Plant source code, Terminal Structured Log Stream & Dashboard.
1	Pressure	bar	(300, 400) # tenths of a bar	Plant source code, Terminal Structured Log Stream & Dashboard.
2	Flow	NM^3/h	(500, 1500) # tenths of Nm^3/h	Plant source code, Terminal Structured Log Stream & Dashboard.
3	Alarm	binary	0	Terminal Structured Log Stream (alarm=0)

Concise explanation of a normal Modbus/TCP transaction

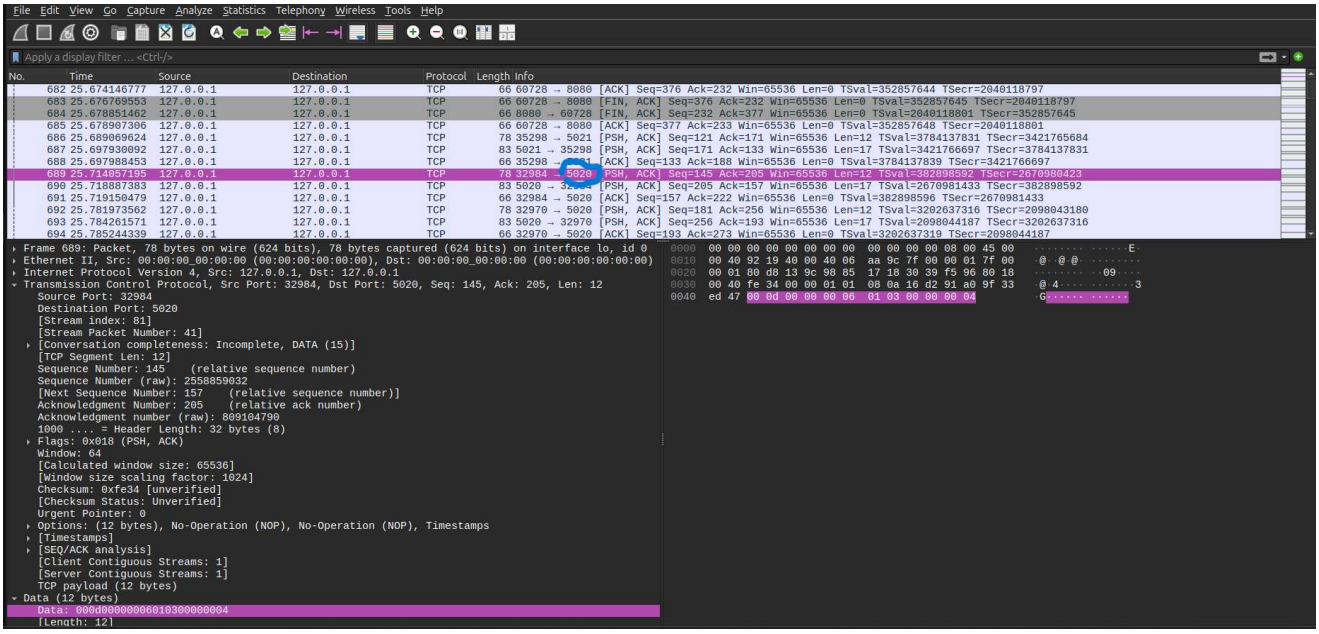
A normal Modbus/TCP transaction begins when a client, such as an HMI or monitoring station, initiates communication by sending an unencrypted request packet over TCP port 502 to a server, such as a PLC. This request package includes an MBAP header and a specific function code that dictates the required operation, such as reading a holding register or writing a coil state. The server receives the packet, processes the command against its internal memory map, and returns a response packet containing the requested telemetry data or execution acknowledgment. Finally, the client receives the response and updates its interface or logs, completing the stateless request-response cycle.

Activity 03 - Packet visibility and protocol analysis

Capture normal Modbus/TCP traffic generated by the supplied simulation and explain what a matched request/response demonstrates.

Expected tasks: Use the Ubuntu guest loopback interface (`lo`) and documented Modbus/TCP port only. Start the normal services and an authorized read client as described in the guide; do not run abnormal demonstrations during this capture. Save a short, labeled PCAP/PCAPNG from your own VM. Identify a request and matching response in Wireshark. Explain the observed endpoint/port, transaction/protocol identifiers, function code, address/value and result; correlate the value with the register map and dashboard. State what the capture proves and what it does not prove, then stop capture and processes.

Evidence: Learner-created PCAP/PCAPNG; annotated request/response screenshots; analysis tied to baseline; capture-method evidence-log entry.



Frame 689 of the baseline packet capture displays an unencrypted Modbus/TCP request transmitted locally from 127.0.0.1 via source port 32984 to destination port 5020. The captured TCP segment carries a payload containing the exact hex string 000100000006010300000004, which executes Function Code 03 to read four contiguous holding registers starting at address 0x0000. This transaction aligns directly with the normal baseline operational profile by demonstrating the local HMI client successfully polling live industrial process telemetry from the plant backend.

Activity 04 - Safe discovery and exposure mapping

Map only the local services present in the supplied source and assess their process relevance without probing external systems.

Expected tasks: Inspect source bind/target configuration and identify only documented loopback endpoints. Prepare a scope card naming VM, loopback target, permitted method, time window, expected state, stop condition and recovery action. Use source inspection and guide-approved read-only checks to verify services; do not widen ranges or use a non-loopback target. Record component role, protocol, process significance and potential consequence of exposure/misuse. Add evidence-based risk-register findings and preventative/detective controls.

Evidence: Approved scope card; local service inventory; relevant command/output or screenshot; risk register; short evidence-based conclusion.

Scope Card

Scoping Parameter	Operational Detail
Virtual Machine (VM)	Local Ubuntu VirtualBox instance hosting the lab environment.
Loopback Target	`127.0.0.1` (Strictly localized to host interface `lo`)
Permitted Method	Only source inspection and the Student Lab Guide's approved read-only checks should be used to confirm which services are active. Do not widen the range, change a target address or scan any non-loopback address.
Time Window	5 Oct 2026 5:42 PM WAT - 15 Oct 2026 11:59 PM WAT
Expected State	Stable telemetry streaming (`temp`, `pressure`, `flow`) with normal baseline values.
Stop Condition	If a target/bind address is not 127.0.0.1, stop and notify the instructor.
Recovery Action	Restore Virtual Machine Baseline Snapshot.

Local Service Inventory

Service/Component	Role	Port	Operational Purpose
Plant	Server	5020	Manages raw core process variables (temperature, pressure, flow) without access control.
Guarded Plant	Guarded Server	5021	Intercepts traffic between client and backend to filter anomalous or unauthorized requests.
Dashboard	Web Interface	8080	Displays real-time plant telemetry and system status without a security monitoring layer.
Guarded Dashboard	Web Interface (Guarded)	8081	Displays secured plant telemetry accompanied by an active Guard Detection Log and status banner.

All services run on lo (127.0.0.1)

Risk Register

Risk ID	Description	Vulnerable Component	Risk Level	Control
RSK-01	Unauthenticated Modbus Write Commands.	Plant Server (127.0.0.1:5020)	Critical	Implement proxy to inspect and filter incoming Modbus payloads.
RSK-02	Cleartext Telemetry Exposure.	Plant Server (127.0.0.1:5020)	High	Redirect logs away from interactive console windows into secure, permission-restricted log files.
RSK-03	Unprotected Dashboard Interface:The undefended HMI lacks authentication or an active security monitoring interface, hiding malicious activity from operators.	Dashboard (127.0.0.1:8080)	High	Transition to the Guarded HMI instance featuring an active Guard Detection Log.
RSK-04	Unrestricted Network Binding	Plant Simulation Runtime.	High	Strict endpoint scoping card enforcement restricting interface bindings exclusively to loopback (`127.0.0.1`)

Conclusion (Visibility vs Safe Operation)

The hydrogen facility simulation underscores a fundamental principle of industrial control systems: operational visibility does not automatically ensure safe operation. While the undefended dashboard successfully provided real-time telemetry and process monitoring, its reliance on unencrypted and unauthenticated Modbus/TCP communication exposed the control path to unauthorized snooping and manipulation. The integration of the security control in the guarded plant established the necessary equilibrium by preserving full operator visibility while actively intercepting and filtering malicious or out-of-bounds commands. Ultimately, achieving true industrial resilience requires pairing clear system observability with robust, proactive defense mechanisms to ensure that operational insights never compromise system integrity or safety.

Activity 05 - Analyze the supplied defensive OT guard

Explain the guard already included in the course source, its observed decisions, audit evidence and limitations; a new guard implementation is not required.

Expected tasks: Inspect `guard.py` and the guarded dashboard in the canonical student source. Create a decision table: each check, input, condition, decision/action and related event/log evidence. Confirm the interpretation from a normal run and approved scenario. Map demonstrated checks to risk-register findings. Discuss limits such as client authentication/authorization, direct-path access, availability, correlated values, threshold tuning and recovery. Do not change the source or thresholds unless the instructor assigns an explicit extension.

Evidence: Guard architecture/decision table; relevant source references; normal-pass/event sample; limitation analysis; updated risk entries.

Guard Architecture/Decision Table

Check Name	Target Input	Trigger Condition	Guard Action	State / Log Output
Absolute Range Check.	Raw write payload values.	Value falls outside hard physical safety boundaries.	Reject.	Drops write entirely; maintains last known-safe value; logs rejection event.
Step Check.	Current write vs. immediate predecessor.	Single write delta exceeds allowable velocity limit.	Reject.	Drops write entirely; blocks sudden spikes; logs rejection event.
Rolling-Baseline Drift Check.	Current accepted value vs. value from DRIFT_WINDOW writes ago.	Cumulative change over the window exceeds drift thresholds.	Reject.	Drops write entirely; successfully blocks slow-ramp cumulative creep; logs alert.
Alarm Register Check.	Register address / function code.	Matches designated alarm or reset register definitions.	Allow.	Forwards write directly without strict numeric constraints.
Periodic Telemetry Mirroring.	Background read from real plant backend.	Background poll values are within absolute safe ranges.	Accept & Mirror.	Updates guarded dashboard state to reflect natural plant drift while blocking out-of-range direct-path corruption.

Relevant Source References (guard.py)

The security proxy's internal mechanics and defensive logic rely on the following implemented structures:

- Rolling-Baseline Drift Window: Configured with DRIFT_WINDOW = 3, shrinking the attacker's operational runway. Cumulative thresholds are defined as DRIFT_THRESHOLD_TEMP = 20 (2.0°C), DRIFT_THRESHOLD_PRESSURE = 20 (2.0 bar), and DRIFT_THRESHOLD_FLOW = 40 (4.0 Nm3/h).
- Accepted Value History (deque): Uses deque(maxlen=DRIFT_WINDOW) to store only accepted values rather than attempted writes, establishing an authoritative baseline of what the plant has actually been allowed to execute.
- Option A Rejection Rule: On any policy violation, the write is dropped entirely. The plant is untouched, local mirrors are unchanged, and the rolling history is not appended, leaving the last known-safe accepted value fully authoritative.
- Direct-Path Isolation via Telemetry Mirroring: Periodic background polling reads the real plant state and updates the guard's exposed dashboard—strictly conditional upon every read falling within absolute safe ranges. This ensures natural environmental drift remains visible on the guarded dashboard while neutralizing unvalidated modifications injected via direct backend port access.

Normal-pass/event Sample

2026-10-15 00:06:56,560 INFO telemetry temp=67.8C pressure=32.6bar flow=88.5Nm3/h alarm=0

2026-10-15 00:06:57,563 INFO telemetry temp=67.9C pressure=32.3bar flow=88.5Nm3/h alarm=0

2026-10-15 00:06:58,564 INFO telemetry temp=68.1C pressure=32.1bar flow=88.7Nm3/h alarm=0

2026-10-15 00:06:59,564 INFO telemetry temp=68.0C pressure=32.3bar flow=89.1Nm3/h alarm=0

2026-10-15 00:07:00,566 INFO telemetry temp=67.8C pressure=32.6bar flow=88.2Nm3/h alarm=0

2026-10-15 00:07:01,581 INFO telemetry temp=67.9C pressure=32.6bar flow=88.5Nm3/h alarm=0

2026-10-15 00:07:02,584 INFO telemetry temp=67.9C pressure=32.8bar flow=88.9Nm3/h alarm=0

2026-10-15 00:07:03,632 INFO telemetry temp=68.0C pressure=32.7bar flow=88.3Nm3/h alarm=0

2026-10-15 00:07:04,633 INFO telemetry temp=68.1C pressure=32.6bar flow=88.9Nm3/h alarm=0

2026-10-15 00:07:05,636 INFO telemetry temp=68.0C pressure=32.7bar flow=88.5Nm3/h alarm=0

Limitation Analysis

While the addition of rolling-baseline drift detection and periodic telemetry mirroring significantly hardens the defense, several inherent structural limitations remain:

- Direct-Path Sub-Threshold Manipulation: If an attacker bypasses the guard by communicating directly with the raw OpenPLC backend port (e.g., port 5021) and injects modifications that stay within absolute safe ranges, the background sync mechanism will reflect those changes only if they pass the absolute range validation. This means subtle, in-bounds manual state corruption could theoretically persist until caught by higher-level logic.
- Threshold Tuning Dependency: Fixed drift thresholds (`DRIFT_THRESHOLD_TEMP = 20`) rely on expected operational velocities. Rapidly fluctuating industrial processes or abrupt legitimate operational adjustments could trigger false-positive rejections if thresholds are too tight.
- Alarm/Reset Register Bypass: Designated alarm or reset registers permit unvalidated passthrough to maintain operational utility. If an attacker identifies and targets vulnerabilities within these exempted registers, the guard will not intercept them.

Updated Risk Entries

Risk ID	Description	Vulnerable Component	Risk Level	Control
RSK-05	Slow-Ramp Cumulative Creep Attacks: Attackers bypass single-step checks by making many tiny, incremental adjustments over time.	Guarded Plant Server (127.0.0.1:5021)	High	Rolling-Baseline Drift Check: Enforce DRIFT_WINDOW = 3 and cumulative delta thresholds on accepted values.
RSK-06	Direct-Path Telemetry Poisoning: Attackers writing directly to the backend port attempt to inject corrupt values into the monitoring dashboard.	Guarded_Dashboard	Medium	Conditional Telemetry Mirroring: Background sync updates guard mirror only if polled values pass absolute safe range validation.

Activity 06 - Controlled security validation

Compare direct and guarded behavior using approved bounded demonstrations supplied with the course source.

Expected tasks: Complete the test plan before execution: source/script, loopback path, baseline, expected outcome, evidence, stop condition and recovery. Select at least three scenarios permitted by the Student Lab Guide/instructor and use supplied parameters unchanged. Run one scenario at a time against the documented direct or guarded path. Never alter target, port, counts, duration or thread settings. Observe script output, dashboard/process state and relevant guard events; compare expected and actual behavior. Verify recovery after each scenario and report variable or unexpected results honestly.

Evidence: Approved test plan; evidence for at least three scenarios; expected-versus-observed comparison; recovery proof for each.

Controlled Validation Test Plan

Scenario ID	Supplied script and local path	Starting state	Expected process/guard behavior	Evidence to collect	Stop condition	Recovery action	Result
SV-01	/course/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/attack_hold.py	Guarded plant simulation running normally on port 5021, stable baseline telemetry.	The script attempts to write a pressure value outside absolute physical limits. The guard is expected to catch the violation via absolute range checks, drops the write entirely, and logs a rejection alert without modifying plant state.	Terminal log showing range rejection error; Wireshark capture of the blocked write command packet.	Script execution finishes attempting the out-of-range write.	None required.	Blocked (Write successfully intercepted and dropped).
SV-02	/course/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/false_data_attack.py	Guarded plant simulation active, baseline telemetry streaming and logging properly.	The script sends batches of out-of-range process values. The guard is expected to evaluate each request, triggers absolute range or drift rejections, drops invalid	Terminal log outputs displaying multiple rejection events; guarded dashboard state showing no unauthorized parameter shifts.	Script finishes sending all programmed batches of invalid process values.	Reset test script state if re-running.	Blocked (Malicious batches neutralized; plant integrity maintained).

			packets, and prevents state corruption on the guarded dashboard.				
SV-03	/course/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/dos_attack.py	Guarded plant simulation running with normal background polling traffic active on port 5021.	The script executes 200 concurrent read requests using 10 threads against the guarded endpoint. The guard is expected to handle the concurrent load, ensuring legitimate reads pass while testing service resilience and stability.	Wireshark capture showing concurrent TCP read segments.	All 10 worker threads complete their assigned 200 read requests.	Verify service stability; restart guard.py if exhaustion or connection drops occurred.	Handled (Operational stability maintained under high concurrency).

Comparison Summary

Scenario	**Baseline state**	**Observed direct/guarded state**	**Log/packet/dashboard reference**	**Finding/limitation**	**Recovery Verified?**
Pressure value outside the simulated range requested.	Guarded plant simulation active, normal baseline telemetry active on guarded path.	Script attempted to write an out-of-range pressure value. Guard intercepted, and dropped the write entirely. The plant state remained unmodified.	Terminal rejection error log and Wireshark capture of blocked write command packet.	Finding: Absolute range checks successfully block hazardous setpoint modifications. Limitation: Relies entirely on pre-defined static safe boundaries.	Yes (no plant recovery needed as state was never corrupted).
Out-of-range process values sent.	Guarded plant simulation active, normal baseline	Script sent batches of invalid, out-of-range	Terminal rejection error log and guarded dashboard	Finding: Multi-batch malicious	Yes (plant backend required no intervention).

	telemetry active on guarded path.	process values. Guard evaluated each batch, triggered absolute/drift rejections, and prevented dashboard corruption.	display showing stable parameters.	payloads are effectively neutralized by proxy logic. Limitation: High-frequency attack scripts can generate excessive log output.	
Demonstration of 200 read requests using 10 threads.	Guarded plant simulation active, normal baseline telemetry active on guarded path.	Script executed 200 concurrent read requests using 10 threads. Concurrent load was processed while maintaining valid read passthrough.	Concurrent TCP read packet trace in Wireshark.	Finding: Proxy handles standard multi-threaded polling loads without crashing. Limitation: Extreme unthrottled thread counts could induce socket exhaustion or latency.	Yes (service remained stable, no manual restart required).

Activity 07 - Recovery, evidence and project defence

Complete cleanup and produce a defensible record of your work using the supplied simulation.

Expected tasks: Stop all services/demonstrations, verify the lab ports are no longer listening, and restore the approved state/snapshot as directed. Complete the evidence log, risk register, test comparison, final report and evidence index using course templates. Cite the course repository and exact source commit; distinguish supplied code from your analysis and any authorized modifications. Prepare a live defence showing normal state, one approved scenario, observed guard response, limitations and verified recovery. Submit a private learner/team evidence repository URL plus matching branch/commit/file references through E-Campus.

Evidence: Final report; repository/commit reference; evidence index/log; risk register; comparison and recovery record; defence plan.

Activity 08 - Supplied-source Modbus/TCP OT security lab

Complete the Ubuntu VM hands-on run of the course source. This is the core practical, unless explicitly waived by the instructor.

Expected tasks: Follow every setup, isolation, dependency, run and cleanup step in the Student Lab Guide. Use only `labs/ot-security/student-lab-source/` and record its commit. The archived copy is not runnable student material. Verify loopback-only configuration, start plant, guard and dashboards, and record a baseline. Complete normal packet analysis and at least three instructor-approved demonstrations one at a time using supplied defaults. Compare direct and guarded results, explain the range/step/drift checks evidenced, describe limitations, and verify cleanup/recovery.

Evidence: Source/version record; topology/register map; baseline and comparison evidence; packet analysis; test plan/results; relevant event evidence; risk entry; limitations and cleanup statement.

6. Minimum acceptance and safety criteria

Criterion	Minimum standard
Source and scope	Correct source path/commit recorded; loopback check passes; no archive execution, bridged access or external target.
Architecture and baseline	Supplied roles and data flow accurately explained; baseline supported by learner observations.
Data model and packet analysis	Register/data-point map traceable to source; normal request/response accurately interpreted.
Exposure and risk	Local service inventory and process risks are evidence-based and within scope.
Guard analysis	Implemented checks and event evidence accurately described; limitations distinguished from recommendations.
Validation	At least three instructor-approved bounded scenarios; expected/observed results, evidence and recovery recorded.
Submission	Private learner evidence repository, final report, evidence log/index and matching commit/file references.

Stop conditions

Stop immediately if any target/bind is not `127.0.0.1`, a real/external system appears, the VM becomes unstable, the approved scope is unclear, or the result is unexpected. Notify the instructor and follow the recovery plan.

7. Evidence items for the LMS

Evidence item	What to submit	Marks mapping
1. Source, scope and architecture	Course source URL/commit; preflight; authorized scope; topology; baseline.	20
2. Data model and normal operation	Register/data-point map; normal dashboard/read evidence; transaction explanation.	15
3. Packet and exposure analysis	Loopback PCAP/PCAPNG; annotated request/response; local inventory; risk register.	15
4. Supplied guard analysis	Decision table/source references; event evidence; limitations/residual risk.	20
5. Validation and recovery	Approved test plan; at least three scenario results; comparison; recovery proof.	15
6. Report and defence	Final report, evidence log/index, private evidence-repo reference and oral/live defence.	15

8. Marking rubric - 100 marks

Criterion	Full-credit standard	Marks
Safe source use, architecture analysis and baseline	Canonical code used safely; correct architecture/data flow; clear normal baseline and reproducible evidence.	20
Modbus/TCP data model and normal-operation analysis	Accurate source-backed register map, roles, normal reads and explanation.	15
Packet visibility, protocol analysis and local exposure	Usable capture; accurate request/response interpretation; scoped service/risk analysis.	15

Criterion	Full-credit standard	Marks
Supplied guard analysis and audit evidence	Correct explanation of implemented checks/events, assumptions and limitations.	20
Controlled validation, findings and recovery	At least three approved tests; evidence-based comparison; verified safe cleanup/recovery.	15
Professional report, evidence and defence	Traceable learner-created evidence; clear attribution; complete report and confident defence.	15
TOTAL	Practical pass threshold: 50%, subject to institutional standard.	100

9. Submission and cleanup checklist

- Use the assigned LMS dates and WAT times; submit before the due time.
- Create a private learner/team repository for your report and permitted evidence; submit the exact repository and matching commit/branch/file URLs.
- Do not commit credentials, tokens, private keys, real site data, or unapproved captures/logs. Do not upload another learner's evidence.
- Stop all lab processes, confirm local listeners have stopped, retain evidence only as authorized and restore the assigned clean snapshot if directed.
- Maximum marks are protected after the first submission. Review the complete package and all links before that submission.